

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

SINGLETONPATTERN

> .vscode

> bin

> lib

> src

App.class

App.java

Logger.class

Logger.java

README.md

src > J Logger.java

```
1 public class Logger {
2
3     // Static instance of Logger
4     private static Logger instance;
5
6     // Private constructor
7     private Logger() {
8         System.out.println("Logger Instance Created");
9     }
10
11     // Public method to get the single instance
12     public static Logger getInstance() {
13
14         if (instance == null) {
15             instance = new Logger();
16         }
17
18         return instance;
19     }
20
21     // Logging method
22     public void log(String message) {
23         System.out.println("LOG: " + message);
24     }
25 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\SingletonPattern> cd "c:\Users\anuja\Desktop\
• } ; if ($?) { java App }
Logger Instance Created
LOG: Application Started
LOG: User Logged In

Checking if both objects are same...
Only one Logger instance exists.
○ PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\SingletonPattern\src>
```

The screenshot displays a Visual Studio Code editor with a project named 'SINGLETONPATTERN'. The Explorer sidebar on the left shows the file structure: `.vscode`, `bin`, `lib`, and `src`. Inside `src`, there are `App.class`, `App.java` (selected), `Logger.class`, `Logger.java`, and `README.md`. The main editor window shows the code for `App.java`:

```
1 public class App {
2     public static void main(String[] args) {
3         Logger logger1 = Logger.getInstance();
4         Logger logger2 = Logger.getInstance();
5
6         logger1.log(message: "Application Started");
7         logger2.log(message: "User Logged In");
8
9         System.out.println();
10
11        System.out.println(x: "Checking if both objects are same...");
12
13        if (logger1 == logger2) {
14            System.out.println(x: "Only one Logger instance exists.");
15        } else {
16            System.out.println(x: "Different Logger instances exist.");
17        }
18    }
19 }
20
```

Below the code editor, the 'TERMINAL' tab is active, showing the command prompt output:

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\SingletonPattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\SingletonPattern\src"
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\SingletonPattern\src> cd .. & if ($?) { java App }
Logger Instance Created
LOG: Application Started
LOG: User Logged In

Checking if both objects are same...
Only one Logger instance exists.
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\SingletonPattern\src>
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Design Pattern > FactoryPattern > J Main.java > Main

```
1  interface Document {
2      void open();
3  }
4
5  // Concrete Document Classes
6  class WordDocument implements Document {
7      public void open() {
8          System.out.println(x: "Opening Word Document...");
9      }
10 }
11
12 class PdfDocument implements Document {
13     public void open() {
14         System.out.println(x: "Opening PDF Document...");
15     }
16 }
17
18 class ExcelDocument implements Document {
19     public void open() {
20         System.out.println(x: "Opening Excel Document...");
21     }
22 }
23
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\FactoryPattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\FactoryPattern"
; if ($?) { java Main }
Opening Word Document...
Opening PDF Document...
Opening Excel Document...
```

```
Design Pattern > FactoryPattern > J Main.java > Main
36 class PdfFactory extends DocumentFactory {
40 }
41
42 class ExcelFactory extends DocumentFactory {
43     public Document createDocument() {
44         return new ExcelDocument();
45     }
46 }
47
48 // Main Class
49 public class Main {
50     Q
51     public static void main(String[] args) {
52
53         DocumentFactory wordFactory = new WordFactory();
54         Document word = wordFactory.createDocument();
55         word.open();
56
57         DocumentFactory pdfFactory = new PdfFactory();
58         Document pdf = pdfFactory.createDocument();
59         pdf.open();
60
61         DocumentFactory excelFactory = new ExcelFactory();
62     }
63 }
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\FactoryPattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\FactoryPattern" & if (\$?) { java Main }
- Opening Word Document...
- Opening PDF Document...
- Opening Excel Document...
- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\FactoryPattern>

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

```
Design Pattern > Builder Pattern > ComputerTest.java > ComputerTest > main(String[])
3    class Computer {
30    public static class Builder {
59        return new Computer(this);
60    }
61 }
62 }
63
64 // Main Class
65 public class ComputerTest {

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\Builder Pattern> cd "c:\Users\anuja\
ava } ; if ($?) { java Computer }

1 error
● PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\Builder Pattern> cd "c:\Users\anuja\
st.java } ; if ($?) { java ComputerTest }
Computer Configuration:
CPU: Intel i5
RAM: 8 GB
Storage: 512 GB
Graphics Card: Integrated

Computer Configuration:
CPU: AMD Ryzen 7
RAM: 16 GB
Storage: 1024 GB
Graphics Card: NVIDIA RTX 4060

○ PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\Builder Pattern> 
```

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

```
Design Pattern > J AdapterTest.java > AdapterTest
1 // Target Interface
2 interface PaymentProcessor {
3     void processPayment(double amount);
4 }
5
6 // Adaptee Class 1
7 class PayPalGateway {
8     public void sendPayment(double amount) {
9         System.out.println("Payment of Rs." + amount + " processed using PayPal.");
10    }
11 }
12
13 // Adaptee Class 2
14 class StripeGateway {
15     public void makeTransaction(double amount) {
16         System.out.println("Payment of Rs." + amount + " processed using Stripe.");
17    }
18 }
19
20 // Adapter for PayPal
21 class PayPalAdapter implements PaymentProcessor {
22
23     private PayPalGateway paypal;
24
25     public void processPayment(double amount) {
26         paypal.sendPayment(amount);
27     }
28 }
29
30 public class AdapterTest {
31     public static void main(String[] args) {
32         PayPalGateway paypalGateway = new PayPalGateway();
33         StripeGateway stripeGateway = new StripeGateway();
34         PayPalAdapter paypalAdapter = new PayPalAdapter();
35         paypalAdapter.paypal = paypalGateway;
36         paypalAdapter.processPayment(1000.0);
37         stripeGateway.makeTransaction(2500.0);
38     }
39 }
40
41 if (args.length > 0) {
42     AdapterTest.main(args);
43 }
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\Builder Pattern> cd "c:\Users\anuja\Desktop\Cognizant
?) { java AdapterTest }
Payment of Rs.1000.0 processed using PayPal.
Payment of Rs.2500.0 processed using Stripe.
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern>
```

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

```
Design Pattern > J DecoratorTest.java > DecoratorTest
1  // Component Interface
2  interface Notifier {
3      void send(String message);
4  }
5
6  // Concrete Component
7  class EmailNotifier implements Notifier {
8
9      @Override
10     public void send(String message) {
11         System.out.println("Sending Email: " + message);
12     }
13 }
14
15 // Abstract Decorator
16 abstract class NotifierDecorator implements Notifier {
17
18     protected Notifier notifier;
19
20     public NotifierDecorator(Notifier notifier) {
21         this.notifier = notifier;
22     }
23 }
24
25 // Concrete Decorator
26 class SmsNotifierDecorator implements NotifierDecorator {
27
28     @Override
29     public void send(String message) {
30         this.notifier.send(message);
31     }
32 }
33
34 // Concrete Decorator
35 class SlackNotifierDecorator implements NotifierDecorator {
36
37     @Override
38     public void send(String message) {
39         this.notifier.send(message);
40     }
41 }
42
43 // Main Class
44 class DecoratorTest {
45
46     public static void main(String[] args) {
47         Notifier notifier = new EmailNotifier();
48         NotifierDecorator decorator = new NotifierDecorator(notifier);
49         decorator.send("Your order has been placed.");
50         decorator.send("Your payment is successful.");
51         decorator.send("Your payment is successful.");
52         decorator.send("Server is running normally.");
53         decorator.send("Server is running normally.");
54         decorator.send("Server is running normally.");
55     }
56 }
57
58 PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\" ; if ($?) {
ratorTest }
Sending Email: Your order has been placed.
Sending Email: Your payment is successful.
Sending SMS: Your payment is successful.
Sending Email: Server is running normally.
Sending SMS: Server is running normally.
Sending Slack Message: Server is running normally.
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> 
```

Exercise 6: Implementing the Proxy Pattern

Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

```
Design Pattern > J ProxyTest.java > ProxyTest
1 // Subject Interface
2 interface Image {
3     void display();
4 }
5
6 // Real Subject
7 class RealImage implements Image {
8
9     private String fileName;
10
11     public RealImage(String fileName) {
12         this.fileName = fileName;
13         loadFromServer();
14     }
15
16     // Simulate loading image from remote server
17     private void loadFromServer() {
18         System.out.println("Loading " + fileName + " from remote server...");
19     }
20
21     @Override
22     public void display() {
23         loadFromServer();
24     }
25 }
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\" ; if (\$?) {
t }
First Display:
Loading Nature.jpg from remote server...
Displaying Nature.jpg

Second Display:
Displaying Nature.jpg
○ PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern>

Exercise 7: Implementing the Observer Pattern

Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified

```
Design Pattern > J ObserverTest.java > ObserverTest
71 public class ObserverTest {
73     public static void main(String[] args) {
81         stockMarket.registerObserver(mobile);
82         stockMarket.registerObserver(web);
83
84         // First update
85         stockMarket.setStock(stockName: "TCS", price: 3500);
86
87         // Second update
88         stockMarket.setStock(stockName: "Infosys", price: 1450);
89
90         // Remove one observer
91         stockMarket.removeObserver(web);
92
93         // Third update
94         stockMarket.setStock(stockName: "Wipro", price: 500);
95     }
}

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - Design Pat

PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\" ; if ($?) { javac ObserverTest.j
verTest }

Stock Updated:
TCS = Rs.3500.0
Mobile App: TCS price updated to Rs.3500.0
Web App: TCS price updated to Rs.3500.0

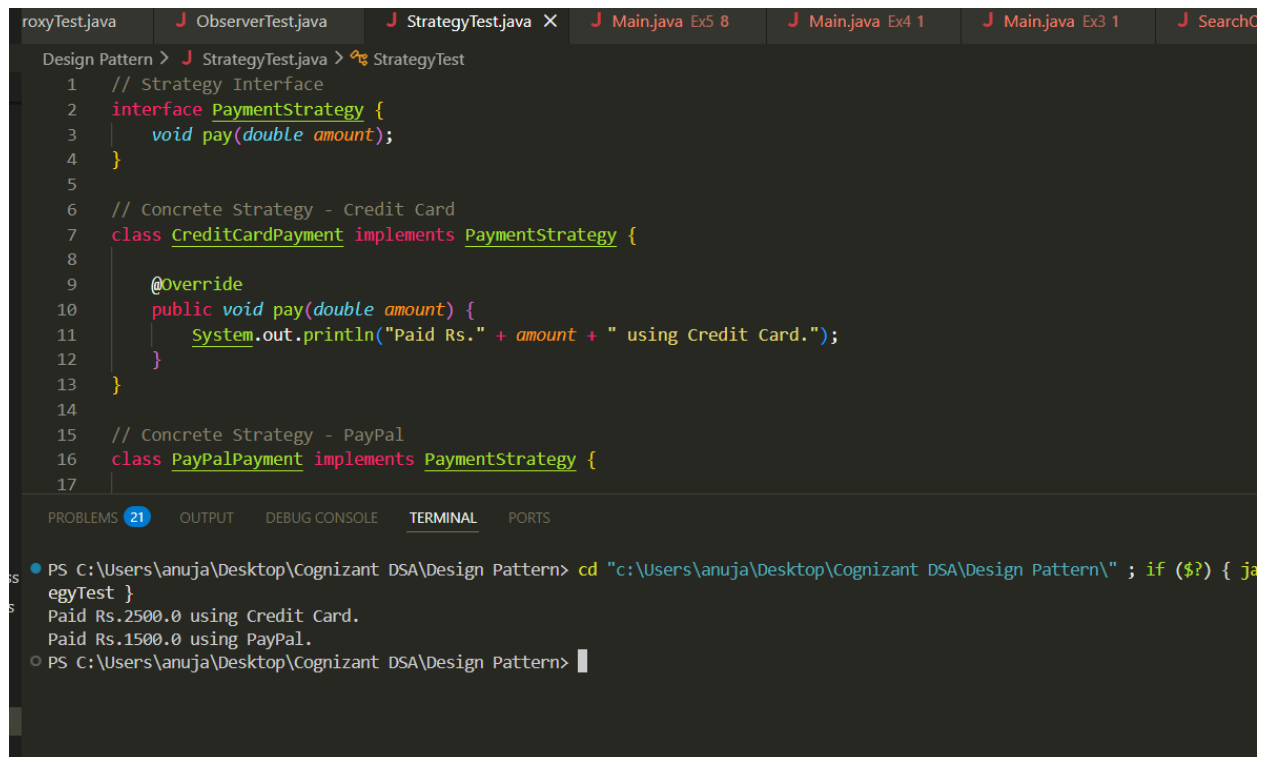
Stock Updated:
Infosys = Rs.1450.0
Mobile App: Infosys price updated to Rs.1450.0
Web App: Infosys price updated to Rs.1450.0

Stock Updated:
Wipro = Rs.500.0
Mobile App: Wipro price updated to Rs.500.0
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern>
```

Exercise 8: Implementing the Strategy Pattern

Scenario:

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.



```
roxyTest.java | J ObserverTest.java | J StrategyTest.java X | J Main.java Ex5 8 | J Main.java Ex4 1 | J Main.java Ex3 1 | J SearchO

Design Pattern > J StrategyTest.java > StrategyTest
1 // Strategy Interface
2 interface PaymentStrategy {
3     void pay(double amount);
4 }
5
6 // Concrete Strategy - Credit Card
7 class CreditCardPayment implements PaymentStrategy {
8
9     @Override
10    public void pay(double amount) {
11        System.out.println("Paid Rs." + amount + " using Credit Card.");
12    }
13 }
14
15 // Concrete Strategy - PayPal
16 class PayPalPayment implements PaymentStrategy {
17
```

PROBLEMS 21 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
● PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\" ; if ($?) { ja
egyTest }
Paid Rs.2500.0 using Credit Card.
Paid Rs.1500.0 using PayPal.
○ PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> |
```

Exercise 9: Implementing the Command Pattern

Scenario: You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

Design Pattern > J CommandTest.java > CommandTest

```
1 // Command Interface
2 interface Command {
3     void execute();
4 }
5
6 // Receiver Class
7 class Light {
8
9     public void turnOn() {
10         System.out.println(x: "Light is ON");
11     }
12
13     public void turnOff() {
14         System.out.println(x: "Light is OFF");
15     }
16 }
17
18 // Concrete Command - Turn ON
19 class LightOnCommand implements Command {
20
21     private Light light;
22
23     public LightOnCommand(Light light) {
24         this.light = light;
25     }
26 }
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern\" ; if (\$?) { javac CommandTest.java }
Light is ON
Light is OFF
- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern>

Exercise 10: Implementing the MVC Pattern

Scenario:

You are developing a simple web application for managing student records using the MVC pattern.

```
Design Pattern > J MVCTest.java > MVCTest
83 public class MVCTest {
84     public static void main(String[] args) {
85
86         StudentView view = new StudentView();
87
88         // Create Controller
89         StudentController controller = new StudentController(student, view);
90
91         // Display Initial Details
92         System.out.println(x: "Initial Student Details:");
93         controller.updateView();
94
95         System.out.println();
96
97         // Update Student Details
98         controller.setStudentName(name: "Priya");
99         controller.setStudentId(id: 102);
100        controller.setStudentGrade(grade: "A+");
101
102        // Display Updated Details
103        System.out.println(x: "Updated Student Details:");
104        controller.updateView();
105    }
106}
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pattern"
Initial Student Details:
Student Details
Name : Anuja
ID : 101
Grade : A

Updated Student Details:
Student Details
Name : Priya
ID : 102
Grade : A+
PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern>
```

Exercise 11: Implementing Dependency Injection

Scenario:

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

Design Pattern > J DependencyInjectionTest.java > DependencyInjectionTest

```
1 // Repository Interface
2 interface CustomerRepository {
3     String findCustomerById(int id);
4 }
5
6 // Concrete Repository
7 class CustomerRepositoryImpl implements CustomerRepository {
8
9     @Override
10    public String findCustomerById(int id) {
11        return "Customer ID: " + id + ", Name: Anuja";
12    }
13 }
14
15 // Service Class
16 class CustomerService {
17
18     private CustomerRepository repository;
19
20     // Constructor Injection
21     public CustomerService(CustomerRepository repository) {
22         this.repository = repository;
```

PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> cd "c:\Users\anuja\Desktop\Cognizant DSA\Design Pat
java DependencyInjectionTest }
Customer ID: 101, Name: Anuja
- PS C:\Users\anuja\Desktop\Cognizant DSA\Design Pattern> |